```
    </>
  )
}

export default Venue
```

I've moved the call to create the subscription out of `useEffect` for kind of quirky logistical reasons—`useEffect` happens after the component loads, whereas we want this subscription to be created before the rest of the component loads so that it can be passed down the React component tree. However, we don't want the subscription to be created more than once, so we create a variable outside the `useEffect` call to store the subscription and then only create the subscription if it does not already exist.

The actual creation of the subscription is only slightly different from what we did in the Stimulus ActionCable example. Rather than have the first argument to `create` be the name of the channel, the first argument is a JavaScript object that has a `channel` attribute. That attribute determines which channel we are connecting to; every other attribute gets passed to the `subscribed` method as part of the `params` hash. As we saw earlier in our definition of the server-side channel, we need to pass the concert ID so that we can specify the name of the channel. In response to the data, in the `received` part of the code we call the same `setVenueData` method we were already calling in response to this data when it was coming from an HTTP request.

The place where the client side sends data back to the ActionCable channel happens in the `Row` component, so we need to pass the `subscription` to the parameters of the `VenueBody` and then to the `Row`. The `VenueBody` only changes to accommodate importing ActionCable, receiving the subscription as a parameter, and passing it along to each row.

The `Row` component changes to receive the parameter and then there's a change to the `onSeatChange` method as well:

chapter_09/04/app/javascript/components/row.tsx

```
function onSeatChange(seatNumber: number): void {
```